Formal Methods: Introduction

Level 06 Semester 1
Core Module for SE

Module Introduction

Module Outline:

Delivery

- 2 hours of Lecture Weekly
- 2 hours of Tutorials Weekly

Independent Study: 152 hours per Semester

Instructions:

- Attend all Lectures and Tutorials
- Questions- Ask module-related questions during lectures and tutorials
- Try Formative Assessment Weekly

Module Introduction

Module Team:

Module Leader & Lecturer- Subhaka Bhavanishankar (subhaka.b@iit.ac.lk)

Module Team – Tutorials

- Pasindu Madhuranga
- Uthpala Sooriyarachchi
- Tharushi Amarabandu
- Dilrushi Pavithra
- Harini Rodrigo

Module Introduction

Assessment:

Summative Assessment:

- Assessment 1 In-Class-Test (ICT) – 50%
- Assessment 2 Coursework(CW) – 50%

Formative Assessment:

- Weekly ICT

Weekly Schedule

Week	Lecture	Tutorials
Week 1	Introduction to Formal Methods	No Tutorials
Week 2	Sets, Constants and Variables	Sets, Variables and Constants
Week 3	Relations	Sets, Variables and Constants
Week 4	Functions	Relations
Week 5	Logic	Relations and Functions
Week 6	Engagement Week	Engagement Week
Week 7	Sequence	Logic
Week 8	Structuring B Specification	Sequence
Week 9	Hoare Logic	Structuring
Week 10	Revision for ICT	Revision for ICT
Week 11	No Lectures	No Tutorials

Required Software

- AtlelierB – AppsAnywhere , Official Website
- ProB – AppsAnywhere , Official Website

If you download from the official website

Active Tcl- Active Tcl Website
Graphviz for visualizing graphs in **.dot** format, available from graphviz.org

Lecture 1: Introduction; the B-Method

The aim of this lecture is to:

- Explain the need for Formal Methods
  - The issue with software bugs,
  - why formal methods are a good or necessary solution
- Provide an overview of the B-Method:
  - outline the main structure – Abstract Machines,
  - present simple examples.

Impact of Poor Software

- Software controls virtually all our systems and activities.
- Software bugs can be a **major** issue.
- The consequences could be
  - a simple nuisance, e.g. laptop crashes,
  - financial loss, e.g. unauthorised financial transfers,
  - loss of property, e.g. Ariane 5 rocket explosion 1996,
  - **loss of life**, e.g. Therac-25 radiotherapy machine, self-driving vehicles

Lessons from Complex Software Projects

Experience shows that **much of the cost** of a complex software projects is spent on **fixing errors**.
Most of these errors are:

- Introduced **early**, during requirements, specification and design stages.
- Only found during implementation, testing or maintenance

Errors are usually caused by:

- lack of precision at the requirements stage,
- incomplete or omitted specification stage,
- making poor design decisions

Attempted Solution: Natural Language

The first software development techniques were pretty ad-hoc and very informal

- Natural language approach or
- Some structured subset to try to improve precision

But natural language is ambiguous, imprecise, etc.

- Consider the variety of programs you can produce from the same coursework specifications

Partial solution: UML

A widely used Software Engineering methodology is **UML**

- Based on **"non-formal structured"** diagrams,
- breaking down the problem into **sub-systems (objects)**,
- capturing aspects like
  - the relationship between **data**
  - the **flow of information** through a system.

Showing correctness relies on extensive **testing**.

Limitations of testing

- Thorough testing is **good** for reducing bugs
- It cannot ensure their **absence**.
- Example:
  - Post-mortem formal analysis for Windows XP
  - Microsoft found lots previously undetected errors including 10000s of null pointer exceptions
  - This despite years of updates with 100s of millions of users
- This is acceptable for some systems, but **not** all of them

Safety and Security Critical Systems

The **Safety and Security Critical Systems** sectors of the software industry has are several recognised **certification standards** for software quality:

- **Evaluation Assurance Level** (EAL1 – EAL7): for the **security** of information systems, e.g. banking sector.
- **Safety Integrity Level** (SIL1 - SIL4): for the **safety** of railway systems, automotive, chemical systems, etc. SIL4 **"Mean Time to Failure"** is about **100,000 years**.

For the highest levels of certification, **"formal methods"** are either essential or legally required.
Companies that produce this type of software (e.g. Siemens, Quinetic) have to use formal methods to **guarantee its quality**.

Formal Specifications

Before we can **check** if software is correct, we first have to **define** what that means

This description will be some kind of **specification** which is

- More abstract than actual code
  - Data specified using mathematical structures like **sets**
  - Focusing on **what** an operation should do, not **how**
- More detailed than e.g. UML
  - Describes **properties** which the data should always satisfy
  - Decribes what **changes** operations make to the data
- Precise, using logic and mathematics

Formal Specifications: The B Method

B-specifications consist of **Abstract Machines (AM)**. It is a concept similar to **classes** in programming.

- Encapsulates a portion of system data and operations.

An abstract machine is a model of:

- **data**: what (part of) a system should be like, and
- **behaviour**: what it should do (operations).

An abstract machine has a:

- **name** to let the rest of the system refer to it,
- **local state**, represented by its variables,
- **interface**, i.e. operations to access and update the state.

Formal Specifications: The B Method

The B method uses a language combining

- **mathematical** concepts and notation:
  - set theory,
  - predicate logic,
  - relations, functions, sequences. . .
- **programming language** constructs:
  - Operations with arguments and return values
  - Conditionals
- **structuring** mechanisms:
  - Multiple machines
  - Relations similar to aggregation/inheritance

B Machine Structure

Abstract machines have several main parts, though not all parts are always present.
(Don't worry about notation yet, we will explain it soon)

- SETS:
  Defines entirely new types of values, e.g.
  Days = {Mon, Tue, Wed , Thu, Fri , Sat , Sun}
  (similar to enums)
- CONSTANTS and PROPERTIES:
  Define derived types based on existing ones, e.g.
  Weekend <: Days & Weekend = {Sat , Sun}
  badDay : Days $*$ NAT & badDay = (Fri,13)
  where we used the built-in set NAT (of natural numbers)

B Machine Structure

Abstract machines have several main parts, though not all parts are always present.
(Don't worry about notation yet, we will explain it soon)

- VARIABLES: Defines the machine's variables
- INVARIANT: Defines the variable properties, e.g.
  spend : NAT &budget : NAT &spend <= budget
- INITIALISATION: Given initial values, e.g.

  spend := 0 || budget := 1000
- OPERATIONS:
  Contains the operations which can query or change the variable values

B Machine Example 1

```
MACHINE FamilyMeeting
 SETS
      Family = {Amber, Ben, Claudia, Derek, Emily}
 VARIABLES
      Invited
 INVARIANT
      Invited <: Family
 INITIALISATION
      Invited := {}
 OPERATIONS
      addInvite( new ) =
           PRE new : FAMILY
           THEN Invited := Invited \/ { new }
      END
END
```

B Machine Example 2

```
MACHINE Sieve
    CONSTANTS
        maxNumber
    PROPERTIES
        maxNumber : NAT    & maxNumber  = 1000000
    VARIABLES
        sieve, primes
    INVARIANT
        sieve <:NAT & primes<:NAT & sieve/\ primes  = {}
    INITIALISATION
        sieve:= 2..maxNumber || primes := {}
    OPERATIONS
        nextPrime= PRE  sieve/= {} THEN
            primes := primes \/ min(sieve) ||
            sieve := {nn|nn : sieve & nn mod min(sieve)/=0 }
        END
END
```

Tools and notation

- We will use two tools to write specifications:
  - Atelier B, an editor and type checker
  - ProB, a Simulator
- These will use an ASCII based notation for mathematical symbols
  - For example, the **union** $A \cup B$ of two sets will be written A \/ B
  - Atelier B has a sub-window for browsing this notation There
  - is a pdf with the notation on blackboard (and you get to use it in the ICT)

Sets

- Sets are essentially "bags" of values.
  - No inherent order
  - Duplicates don’t count: any value is either in the set or not
  - For example $\{1,2\} = \{2,1\} = \{1,1,2,1,2\}$
- They are a fundamental structure in maths
  - All maths can be defined in terms of sets
  - Additional structures (orderings, functions, . . .) are also sets
  - But it can be more convenient to assume other pre-defined types like numbers

Sets in mathematics

- Sets can contain essentially anything.
  - Colours = {red,green,blue }
  - SmallPrimes = {2, 3, 5, 7}
  - Nested sets: Multiples = {{2,4,6,8}, {3,6,9}, {5}, {7}}
    - Note that {2}, {{2}}, {{{2}}} etc are all **different** things
- Sets used in the B method cannot be "mixed" like {green, 8, {} }
  - This restriction does not exist in more general theories
  - But it makes it easier to reason about the sets

Set operations

There are a number of common operations we can apply to sets

Operation	Mathnotation	ASCIInotation
Union	$A \cup B$	A\/B
Intersection	$A \cap B$	A/\B
Setdifference	$A \setminus B$	A - B
Number of elements	$\lvert A \rvert$	card(A)

Example: If A = {1,2,3,4,5} and B = {1,4,9} then

- A \/ B = {1,2,3,4,5,9}
- A /\ B = {1,4}
- A - B = {2,3,5}
- B-A={9}
- card(A) = 5 and card(B) = 3

Set operations

There are a number of common operations we can apply to sets

Operation	Mathnotation	ASCIInotation
Member	$x \in A$	x : A
Non-member	$x \notin A$	x/:A
Subset	$A \subseteq B$	A<:B
Strict subset	$A \subset B$	A<<:B
Not a subset	$A \not\subseteq B$	A/<:B
Not a strict subset	$A \not\subset B$	A/<<:B

Example: If A = {1,2,3,4,5} and B = {1,4} then

- 3:A but 3/:B
- A /<: B (and therefore also A /<<: B)
- B <<: A (and therefore also B <: A) A
- A <: A but A /<<: A

Set definitions

New sets can be defined in a number of ways:

- The empty set {} (sometimes written ∅)
- Explicit enumerations like {1, 2, 5}
- Integer ranges like 2..7 (meaning {2, 3, 4, 5, 6, 7})
- Power set (set of subsets) POW(A)
  - Example: if A = {1,2,4} then
    POW(A) = {{},{1},{2},{4},{1,2},{1,4},{2,4},{1,2,4}}
  - Mathematical notationis $\mathbb{P}(A)$ or 2^A
- Cartesian product (set of pairs) A*B
  - Example: if A = {red, green} and B = {1,2,3} then
    A*B = {(red,1),(red,2),(red,3),(green,1),(green,2),(green,3)}
- Set comprehension {x | x:A & condition}
  - Example: {x | x : NAT & x < 4} = {1,2,3}

These can be combined, e.g. {A | A : POW(NAT) & 2 : A}

Numbers

The integers and their subsets have the usual arithmetic operations

Definition Integers	Mathnotation	ASCIInotation
Naturalnumbersfrom0	Z	INTEGER
Naturalnumbersfrom1	N	NAT
Addition Subtraction	N1	NAT1
Multiplication Division	x+y	x+y
Remainder Power	x - y	x - y
	x ∗ y	x * y
	x/y	x/y
	xmody	x mod y
	x^y	x**y

Numbers

There integers and their subsets have the usual arithmetic operations

Definition	Mathnotation	ASCIInotation
Previousnumber	pred(x)	pred(x)
Nextnumber	succ(x)	succ(x)
Equality	x=y	x=y
Inequality	x≠y	x/=y
Less	x<y	x<y
Lessorequal	x ≤y	x<=y
Greater	x>y	x>y
Greaterofequal	x ≥y	x>=y

In Class Set Exercises

Given the following sets:

$A = \{1,2,3,4,5,6,7,8\}$ $B = \{1,4,7,9,10\}$

$C = \{2,4,6\}$ $D = 1..12$

(1) Evaluate the following set expressions:

$card(B)$ $B \vee C =$

$C \setminus 1..5 =$ $A - B =$

$POW(C) =$

(2) Using **set comprehension** define the set of even numbers less than 20.

(3) Determine which of the following are true or false:

$5 : D$ $4 : B$

$A <: D$ $B <: A$

$C <<: B$ $\{2, 4\} : POW(C)$